

Namespace

기존에 c는 순차구조였다. 그러다 보니깐 예전에 알약 디스펜서를 만들었을 때 디스펜서가 알약을 배출하는 동안은 알약을 몇 개 배출할지에 대한 정보를 받지 못하였다. 하지만 namespace를 이용해서 다른 헤더 파일에 있는 함수들을 불러 올 수 있다고 안다. 그렇게 된다면 다른 파일로 넘어가서 그 파일에서 코드가 실행된다면 기존에 main스트림하고 또 다른 스트림이 만들어 지는 것은 아닐까? 하는 생각이 들었다. 예를 들자면 pill_dispensor1이라는 객체의 생성자에서 input_pill_num1이라는 다른 파일의 객체를 실행한다. 그 객체의 생성자에서 while문을 이용해 계속 배출시킬 알약의 개수를 계속 입력받고 다시 pill_dispensor1의 pill_num이라는 변수에 값을 전달한다면, 기존의 알약디스펜서 코드, 알약갯수 입력받는 코드 이렇게 두개가 병렬적으로 코드가 동작할 수 있는 것은 아닐까라는 생각이 들었다. 이렇게 된다면 한 파일에서 여러 개의 병렬적인 코드가 진행되고 더 많은 움직임을 동시에 제어할 수 있으므로 다양한 방법을 시도할 수 있게 되지 않을까라고 생각하였다.

생성자&소멸자

과제 1번에서도 말하였지만 객체를 선언할 때 생성자와 소멸자를 선언할 수 있다. 이는 객체 이름과 같은 함수를 말하며, 소멸자는 ~를 앞에 붙인다. 생성자를 이용하여서 각각의 모터를 객체로 관리할 때 init이라는가 초기의 값을 이곳에서 선언하여 관리하기 편하기 편할 것 같다.

또한 소멸자는 과제1,2에서는 동적할당을 해제하는 기능밖에 없었다. 하지만 Ros2세미나에서 로봇의 디버깅을 위한 데이터를 전송하는 것을 실시간으로 화면에 띄우거나 usb로 전송받기 보다는 로봇이 동작을 완전히 멈추고 마지막에 전송받아 디버깅하는 것이 로봇이 동작하는데 간섭이 덜 생길것이라는 것을 배웠다. 그래서 따로 함수로 디버깅 데이터를 항상 받는 것 보다는 소멸자에서 디버깅데이터 전송을 하여 로봇의 삽질에 대한 시행착오를 줄어 보는 것이 어떨까라고 생각이 들었다.

또한 소멸자에 이 객체가 종료할 때 꼭 해야하는 것들을 적어놓는 곳이라고 느껴져서 이러한 생각을 써보았다.

멤버 접근 제어

public은 말그대로 객체 외부에서도 이 멤버를 접근할 수 있는 것, private는 객체 내부에서만 접근할 수 있는 것이다. 좀 더 쉽게는 객체 내의 함수에서만 주로 사용하는 것이다. 근데 만약 public으로 private에 접근하려고 하면 가능할까가 궁금하였다. 또한 참조자로 public이 private를 참조한다면 외부에서 사용가능할까에 대한 의문이 생겼다.

ai에게 물어보니 두 경우 모두 접근할 수 있다고 한다. 전자는 애초에 public 함수에서 private변수를 사용하니깐 문제가 되지 않는다고 하였다. 덧붙여서 이것이 가능한 이유는 private이라고 데이터 자체에 자물쇠가 걸린 것이 아니기 때문이다. 다시 생각해 보니 굉장히 타당한 답변이었다. 왜냐하면 만약 코드를 내가 작성한다고 하면 데이터 (위치)자체에 락을 거는 것 보다는 이용의 편의성에 의한 private과 public 구분이 주목적으로 할 것 같다. 그외에 private 데이터에 대한 보안성을 이유로 코드에 접근하지는 않을 것이기 때문이다.

로봇적인 접근은 아니지만 이것을 이용해서 개발자들이 주로 쓰는 백도어를 의도적으로 만들 수도 있지 않을까? 생각이 들었다. 소스파일자체에서 코드를 수정해서 디버깅을 해볼 수도 있지만 main 파일에서 간단하고 가시성이 좋게 private변수에 대한 수정을 하고 싶다면 그것을 public으로 돌리지 않고 참조만 하여서 수정을 해보면 어떨까라고 생각을 해보았다.

Overloading

함수의 이름이 중복되어 사용되는 것으로 매개 변수 타입이 다르거나 개수가 다르면 성공한다. 예를 들면 move(int distance), move(int time, int speed) 이런 느낌으로 모터가 동작하는 함수 2개가 있다고 해보면, 단순히 거리만 알려주고 함수를 동작시키고 싶을때와 결과값은 예측할 수 없지만 시간과 pwm값만 넣으면 모터가 동작하게 하고 싶을 때, 이렇게 구분해서 모터를 제어하는 두가지 함수를 만들 수 있다. 그럴때 move1, move2이렇게 함수를 선언하는 것 보다는 move로 묶지만 매개변수만 다르게 해서 받으면 훨씬 함수를 관리하기 편리해 진다. 이것을 나중에 터틀봇 할 때 사용해 볼려고 한다.

Call by reference/value

Reference는 변수의 주소값을 참조해서 사용하는 것이다. 이것을 이용하면 변수를 사용할 때 따로 변수를 다시 만들지 않아도 되어서 메모리 용량을 줄일 수 있다는 것이다. 라인트레이서나 최종프로젝트를 할때는 그정도의 메모리를 사용하지는 않았지만 터틀봇만 하더라도 카메라와 라이다(이번 프로젝트에서는 사용하지 않지만)를 사용한다.

Atmega를 사용하지 않고 jetson을 사용하는 것이 이 센서들 때문일 것이라고 생각하기 때문에, 데이터 값을 call by value로 하기보다는 reference로 하는 것이 메모리 관리하는데 효과적일 것 같다.